Advanced Programming Project Documentation

A.Y. 2025/2026

Version 1.0

Common Crate Specifications & Protocols

Contents

1 Introduction **1**

2 System Architecture **1**
2.1 Crate Structure . . . 1
2.2 Component Relationships . . . 2
2.3 Guiding Principles . . . 2

3 Core Components **2**
3.1 Forge . . . 2
3.1.1 Interface . . . 2
3.1.2 Protocol Rules . . . 3
3.1.3 Source Reference . . . 3
3.2 Rocket . . . 3
3.2.1 Interface . . . 3
3.2.2 Construction Protocol . . . 3
3.2.3 Source Reference . . . 4
3.3 Sunray . . . 4
3.3.1 Interface . . . 4
3.3.2 Rules . . . 4
3.3.3 Source Reference . . . 4
3.4 Asteroid . . . 4
3.4.1 Interface . . . 4
3.4.2 Rules . . . 5
3.4.3 Source Reference . . . 5
3.5 EnergyCell . . . 5
3.5.1 Interface . . . 5
3.5.2 Energy Consumption Rules . . . 6
3.5.3 Key Constraints . . . 6
3.5.4 Source Reference . . . 6
3.6 Resource System . . . 6
3.6.1 Core Trait . . . 6
3.6.2 Basic Resources . . . 7
3.6.3 Recipes and Combination . . . 8
3.6.4 Complex Resources . . . 8
3.6.5 Complex Resource Requests . . . 9
3.6.6 Utility Types . . . 9
3.6.7 Generator and Combinator . . . 10
3.6.8 Source Reference . . . 10
3.7 Planet . . . 11
3.7.1 Planet AI Trait . . . 11
3.7.2 Planet Types . . . 12
3.7.3 Planet State . . . 12
3.7.4 Planet Construction . . . 13
3.7.5 Construction Validation . . . 13
3.7.6 Planet Execution . . . 14
3.7.7 Source Reference . . . 14

3.8 Explorer . . . 14
3.8.1 Core Responsibilities . . . 14
3.8.2 Resource Storage . . . 15
3.8.3 Key Constraints . . . 15

4 Message Enum Definitions **15**
4.0.1 Message Flow Rules . . . 15
4.1 Orchestrator ↔ Planet Protocol . . . 16
4.1.1 Message Definitions . . . 16
4.1.2 Message Response Expectations . . . 17
4.2 Explorer ↔ Planet Protocol . . . 17
4.2.1 Message Definitions . . . 17
4.3 Orchestrator ↔ Explorer Protocol . . . 18
4.3.1 Message Definitions . . . 18
4.3.2 Movement Coordination Protocol . . . 19
4.4 Source Reference . . . 19

5 Protocol Interaction Stories **20**
5.1 Lifecycle Management . . . 20
5.2 Sunray and Asteroid Interactions . . . 22
5.3 Resource Operations . . . 23
5.4 Explorer Navigation . . . 26
5.5 State Discovery and Inspection . . . 27
5.6 Energy Management . . . 27
5.7 Bag Contents Request . . . 28

6 Observability and Diagnostics **29**
6.1 Unified Logging Framework . . . 29
6.2 Log Structure and Types . . . 29
6.3 Integration with Rust's Log Ecosystem . . . 30
6.4 Logging Requirements for Implementers . . . 30
6.5 Source Reference . . . 30

7 User-Inconstructible Types **30**
7.1 Enforcement Mechanisms . . . 30
7.2 Technical Enforcement . . . 31

8 Implementation Guidelines **31**
8.1 For Planet Implementers . . . 31
8.2 For Explorer Implementers . . . 31
8.3 For Orchestrator Implementers . . . 31
8.4 Common Requirements . . . 32

9 Source Code References **32**

10 Protocol Summary **33**
10.1 Key Invariants . . . 33
10.2 Error Handling Philosophy . . . 33
10.3 Extension Points . . . 33

1 Introduction

This document defines the shared architecture, specifications, and communication protocols for the Advanced Programming course project. Its purpose is to enable independent groups to develop compatible components by providing:

- **Consistent interfaces** for all game entities
- **Unambiguous protocols** for inter-component communication
- **Rigorous specifications** of component behavior
- **Safety guarantees** through user-inconstructible types
- **Compatibility standards** for cross-group integration

This documentation serves as the authoritative reference for external implementers who need to create planets, explorers, or other components that interoperate with existing systems. The focus is on public interfaces and protocols, not internal implementations.

2 System Architecture

2.1 Crate Structure

The common crate provides the foundation for all implementations:

```
common/
 src/
    lib.rs              # Public exports
    components/         # Game entity definitions
       asteroid.rs      # Asteroid struct
       energy_cell.rs   # EnergyCell with charging logic
       forge.rs         # Singleton Forge
       planet.rs        # Planet AI trait & types
       resource.rs      # Resource types & recipes
       rocket.rs        # Rocket struct
       sunray.rs        # Sunray struct
    protocols/          # Communication protocols
        messages.rs     # All message enums
    protocols.rs
    components.rs
    logging.rs
 Cargo.toml
```

2.2 Component Relationships

The system follows a strict hierarchical architecture:

- **Orchestrator**: Central coordinator owning a singleton Forge
- **Planets**: Autonomous entities with AI behavior and resource capabilities
- **Explorers**: Mobile agents that traverse between planets
- **Forge**: Singleton component creating Sunrays and Asteroids
- **Communication**: All interactions via defined message protocols

2.3 Guiding Principles

- **Protocol-first design**: Interfaces defined by message protocols
- **User-inconstructible types**: Critical types cannot be created by users
- **No undocumented panics**: All failure modes are specified
- **Clippy-pedantic compliance**: Highest code quality standards
- **No unsafe code**: Memory safety guaranteed

3 Core Components

3.1 Forge

Purpose: Singleton component that creates Sunrays and Asteroids. Ensures system-wide uniqueness.

3.1.1 Interface

```rust
/// Singleton that creates Sunrays and Asteroids.
/// Construction fails if a Forge already exists.
pub struct Forge {
    _private: (),
}

impl Forge {
    /// Creates the singleton Forge.
    /// Returns Err if Forge already exists.
    pub fn new() -> Result<Forge, String>;

    /// Creates a new Sunray.
    pub fn generate_sunray(&self) -> Sunray;

    /// Creates a new Asteroid.
    pub fn generate_asteroid(&self) -> Asteroid;
}
```

Listing 1: Forge Public Interface

3.1.2 Protocol Rules

- **Singleton enforcement**: Only one Forge can exist in the entire system
- **Orchestrator requirement**: Every orchestrator must own a Forge (not borrow)
- **Exclusive creation**: Only Forge can create Sunrays and Asteroids

! Critical Requirement: Orchestrator implementations must contain a `Forge` field (not a reference) to prevent multiple orchestrators. This is enforced at runtime.

3.1.3 Source Reference

Complete implementation: forge.rs

3.2 Rocket

Purpose: Planetary defense mechanism against Asteroids.

3.2.1 Interface

```rust
/// Planetary defense against Asteroids.
/// Built using charged EnergyCells.
pub struct Rocket {
    _private: (),
}

impl Rocket {
    /// Constructs Rocket from charged EnergyCell.
    /// Consumes the cell's energy.
    pub fn new(energy_cell: &mut EnergyCell) -> Result<Rocket,
        String>;
}
```

Listing 2: Rocket Construction Interface

3.2.2 Construction Protocol

- **Input**: One fully charged EnergyCell
- **Consumption**: Cell is discharged during construction
- **Availability**: Only planet types A and C can have rockets
- **Error case**: Construction fails with `Err` if EnergyCell is not charged

Deflection Mechanism: Rockets have no inherent behavior. Asteroid deflection is handled by the Planet's `handle_asteroid()` method, which returns an owned Rocket if available for deflection.

3.2.3 Source Reference

Complete implementation: rocket.rs

3.3 Sunray

Purpose: Immutable energy unit used to charge EnergyCells. Created exclusively by the Generator.

3.3.1 Interface

```rust
/// Immutable energy unit.
/// Created only by Generator, consumed by EnergyCells.
pub struct Sunray {
    _private: (),
}
```

Listing 3: Sunray Structure

3.3.2 Rules

- **Single-use**: Each Sunray charges exactly one EnergyCell
- **Consumption**: Sunray is consumed during charging
- **Idempotent charging**: Charging already-charged cell wastes Sunray

3.3.3 Source Reference

Complete implementation: sunray.rs

3.4 Asteroid

Purpose: Destructive object that planets must deflect using Rockets.

3.4.1 Interface

```rust
/// Destructive object targeting planets.
/// Created only by Generator, deflected by Rockets.
pub struct Asteroid {
    _private: (),
}
```

Listing 4: Asteroid Structure

3.4.2 Rules

- **Rocket required**: Planet without Rocket is destroyed
- **Rocket consumption**: Rocket is consumed during deflection
- **Immediate response**: Planet must respond with `AsteroidAck` (with `Some(Rocket)`) if defended, or `KillPlanetAck` if destroyed
- **Termination**: Planet execution terminates after sending `KillPlanetAck`
- **Explorer termination cascade**: All Explorers currently present on the destroyed planet must also be terminated

3.4.3 Source Reference

Complete implementation: asteroid.rs

3.5 EnergyCell

Purpose: Stores energy for resource generation, combination, and rocket construction.

3.5.1 Interface

```rust
/// Stores binary energy state (charged/uncharged).
pub struct EnergyCell {
    charge: bool,
}

impl EnergyCell {
    /// Creates new uncharged cell.
    pub fn new() -> Self;

    /// Charges cell using Sunray.
    /// Wastes Sunray if already charged.
    pub fn charge(&mut self, sunray: Sunray);

    /// Discharges cell for resource operations.
    /// Returns Err if cell not charged.
    pub fn discharge(&mut self) -> Result<(), String>;

    /// Checks if cell is charged.
    pub fn is_charged(&self) -> bool;
}
```

Listing 5: EnergyCell Public Interface

3.5.2 Energy Consumption Rules

Operation	Energy Cells	Effect
Basic Resource Generation	1 charged cell	Cell discharged
Complex Resource Combination	1 charged cell	Cell discharged
Rocket Construction	1 charged cell	Cell discharged
Sunray Charging	1 Sunray	Cell becomes charged

Table 1: Energy Cell Consumption Rules

3.5.3 Key Constraints

- **Binary state**: Charge is either 0 or 1 (maximum charge is one)
- **Initial state**: New cells start uncharged (zero initial charge)
- **Idempotent charging**: Charging already-charged cell wastes Sunray
- **Discharge failure**: Attempting to discharge uncharged cell returns `Err("EnergyCell not charged!")`

3.5.4 Source Reference

Complete implementation: energy_cell.rs

3.6 Resource System

Purpose: Defines all basic and complex resources with strict combination recipes.

3.6.1 Core Trait

```rust
/// Common interface for all resources.
pub trait Resource: std::fmt::Display {
    /// Returns resource name as static string.
    fn to_static_str(&self) -> &'static str;
}
```

Listing 6: Resource Trait

3.6.2 Basic Resources

Four elemental resources that planets can generate. These combine to form complex resources.

Basic Resource
Hydrogen
Oxygen
Carbon
Silicon

Table 2: Basic Resources

```rust
/// Identifies basic resource types.
#[derive(Debug, Clone, Copy, Eq, PartialEq, Hash)]
pub enum BasicResourceType {
    Hydrogen,
    Oxygen,
    Carbon,
    Silicon,
}

/// Contains actual basic resource instances.
#[derive(Debug, PartialEq, Eq, Hash)]
pub enum BasicResource {
    Hydrogen(Hydrogen),
    Oxygen(Oxygen),
    Carbon(Carbon),
    Silicon(Silicon),
}

/// Individual basic resource structs.
pub struct Hydrogen { _private: () }
pub struct Oxygen { _private: () }
pub struct Carbon { _private: () }
pub struct Silicon { _private: () }
```

Listing 7: Basic Resource Types

3.6.3 Recipes and Combination

Recipe Management: Recipes are added to Generator/Combinator via `add()` methods during planet construction. Each recipe can only be added once.

Complex Resource	Recipe	Energy Cost
Water	Hydrogen + Oxygen	1 charged cell
Diamond	Carbon + Carbon	1 charged cell
Life	Water + Carbon	1 charged cell
Robot	Silicon + Life	1 charged cell
Dolphin	Water + Life	1 charged cell
AIPartner	Robot + Diamond	1 charged cell

Table 3: Complex Resource Recipes

3.6.4 Complex Resources

```rust
/// Identifies complex resource types.
#[derive(Debug, Clone, Copy, Eq, PartialEq, Hash)]
pub enum ComplexResourceType {
    Diamond,
    Water,
    Life,
    Robot,
    Dolphin,
    AIPartner,
}

/// Contains actual complex resource instances.
#[derive(Debug, PartialEq, Eq, Hash)]
pub enum ComplexResource {
    Diamond(Diamond),
    Water(Water),
    Life(Life),
    Robot(Robot),
    Dolphin(Dolphin),
    AIPartner(AIPartner),
}

/// Individual complex resource structs.
pub struct Diamond { _private: () }
pub struct Water { _private: () }
pub struct Life { _private: () }
pub struct Robot { _private: () }
pub struct Dolphin { _private: () }
pub struct AIPartner { _private: () }
```

Listing 8: Complex Resource Types

3.6.5 Complex Resource Requests

```rust
/// Encodes requests for complex resource combination.
/// Used in Explorer-Planet communication.
#[derive(Debug, PartialEq, Eq, Hash)]
pub enum ComplexResourceRequest {
    Water(Hydrogen, Oxygen),
    Diamond(Carbon, Carbon),
    Life(Water, Carbon),
    Robot(Silicon, Life),
    Dolphin(Water, Life),
    AIPartner(Robot, Diamond),
}
```

Listing 9: Complex Resource Request Enum

3.6.6 Utility Types

```rust
/// Identifies resource types without containing the resource.
#[derive(Debug, Clone, Copy)]
pub enum ResourceType {
    Basic(BasicResourceType),
    Complex(ComplexResourceType),
}

/// Contains actual resource instances (basic or complex).
#[derive(Debug, PartialEq, Eq, Hash)]
pub enum GenericResource {
    BasicResources(BasicResource),
    ComplexResources(ComplexResource),
}
```

Listing 10: Resource Utility Types

3.6.7 Generator and Combinator

```rust
/// Manages basic resource generation recipes.
pub struct Generator {
    recipes: HashSet<BasicResourceType>,
}
impl Generator {
    /// Generates basic resource using charged cell.
    pub fn new() -> Generator;
    pub fn contains(&self, basic: BasicResourceType) -> bool;
    pub fn all_available_recipes(&self) -> HashSet<BasicResourceType>;

     /// Generates Hydrogen using charged EnergyCell.
    pub fn make_hydrogen(&self, cell: &mut EnergyCell) -> Result<Hydrogen, String>;
    // Similar for Oxygen, Carbon, Silicon
}

/// Manages complex resource combination recipes.
pub struct Combinator {
    recipes: HashSet<ComplexResourceType>,
}

impl Combinator {
    /// Combines resources using charged cell.
    pub fn new() -> Combinator;
    pub fn contains(&self, complex: ComplexResourceType) -> bool;
    pub fn all_available_recipes(&self) -> HashSet<ComplexResourceType>;

    /// Combines Hydrogen and Oxygen into Water.
    /// Returns resources on error for recovery.
    pub fn make_water(&self, h: Hydrogen, o: Oxygen,
                      cell: &mut EnergyCell) -> Result<Water, (String, Hydrogen, Oxygen)>;
    // Similar for other combinations
}
```

Listing 11: Resource Creation Interfaces

Error Handling: Failed combinations return both the error and the input resources, allowing recovery.

3.6.8 Source Reference

Complete implementation: resource.rs

3.7 Planet

Purpose: Stateful game entity with energy management, resource capabilities, and communication endpoints.

3.7.1 Planet AI Trait

Resource Handling Restriction: The PlanetAI implementation **must not** cache or hold resources internally.

```rust
/// Defines planet behavior through message handlers.
pub trait PlanetAI: Send {
    /// Handles orchestrator messages (except Start/Stop/Asteroid).
    fn handle_orchestrator_msg(
        &mut self,
        state: &mut PlanetState,
        generator: &Generator,
        combinator: &Combinator,
        msg: OrchestratorToPlanet,
    ) -> Option<PlanetToOrchestrator>;

    /// Handles all explorer messages.
    fn handle_explorer_msg(
        &mut self,
        state: &mut PlanetState,
        generator: &Generator,
        combinator: &Combinator,
        msg: ExplorerToPlanet,
    ) -> Option<PlanetToExplorer>;

    /// Handles asteroid threats.
    /// Returns Some(Rocket) to survive, None to be destroyed.
    fn handle_asteroid(
        &mut self,
        state: &mut PlanetState,
        generator: &Generator,
        combinator: &Combinator,
    ) -> Option<Rocket>;

    /// Called when planet AI starts.
    fn start(&mut self, state: &PlanetState);

    /// Called when planet AI stops.
    fn stop(&mut self, state: &PlanetState);
}
```

Listing 12: PlanetAI Trait

Important: The `Generator` and `Combinator` parameters refer to `resource::Generator` and `resource::Combinator` (resource recipe managers), not `components::generator::Generator` (Sunray/Asteroid creator).

3.7.2 Planet Types

Type	Energy Cells	Generation Recipes	Rockets	Combination Recipes
A	5 cells	At most 1 type	Allowed	None
B	1 cell	Unlimited types	Not allowed	1 type
C	1 cell	At most 1 type	Allowed	All 6 types
D	5 cells	Unlimited types	Not allowed	None

Table 4: Planet Type Specifications (Constraints apply to number of resource types, not total quantity)

Constraint Clarification: The "bounded/unbounded" rules refer to the **number of different resource types** a planet can generate/combine, not the total quantity of resources. For example, a Type A planet can generate at most 1 type of basic resource, but can generate unlimited quantities of that resource.

3.7.3 Planet State

```rust
/// Represents planet's internal state.
pub struct PlanetState {
    id: u32,
    energy_cells: Vec<EnergyCell>,
    rocket: Option<Rocket>,
    can_have_rocket: bool,
}
impl PlanetState {
    pub fn id(&self) -> u32;
    pub fn cell(&self, i: usize) -> &EnergyCell;
    pub fn cell_mut(&mut self, i: usize) -> &mut EnergyCell;
    pub fn cells_count(&self) -> usize;
    pub fn cells_iter(&self) -> Iter<'_, EnergyCell>;
    pub fn cells_iter_mut(&mut self) -> IterMut<'_, EnergyCell>;
    /// Charges the first empty (discharged) cell.
    /// Returns Some(Sunray) if no cell to charge.
    pub fn charge_cell(&mut self, sunray: Sunray) -> Option<Sunray>;
    /// Returns mutable borrow of first empty cell with index.
    pub fn empty_cell(&mut self) -> Option<(&mut EnergyCell, usize)>;
    /// Returns mutable borrow of first charged cell with index.
    pub fn full_cell(&mut self) -> Option<(&mut EnergyCell, usize)>;
    pub fn can_have_rocket(&self) -> bool;
    pub fn has_rocket(&self) -> bool;
    pub fn take_rocket(&mut self) -> Option<Rocket>;
    pub fn build_rocket(&mut self, i: usize) -> Result<(), String>;
}
```

Listing 13: PlanetState Interface

3.7.4 Planet Construction

```rust
pub struct Planet {
    state: PlanetState,
    planet_type: PlanetType,
    pub ai: Box<dyn PlanetAI>,  // Note: Boxed trait object
    generator: Generator,        // resource::Generator
    combinator: Combinator,      // resource::Combinator
    // ... channels
}

impl Planet {
    pub fn new(
        id: u32,
        planet_type: PlanetType,
        ai: Box<dyn PlanetAI>,  // Box required for type erasure
        gen_rules: Vec<BasicResourceType>,
        comb_rules: Vec<ComplexResourceType>,
        orchestrator_channels: (...),
        explorer_channels: (...),
    ) -> Result<Planet, String>;

    // Accessor methods
    pub fn id(&self) -> u32;
    pub fn planet_type(&self) -> PlanetType;
    pub fn state(&self) -> &PlanetState;
    pub fn generator(&self) -> &Generator;
    pub fn combinator(&self) -> &Combinator;
}
```

Listing 14: Planet Constructor

3.7.5 Construction Validation

The `Planet::new()` method validates:

- `gen_rules` cannot be empty
- Planet types with bounded generation (A, C) accept at most 1 generation recipe
- Combination rules count must not exceed planet type limit (0, 1, or 6)
- Rocket construction only allowed for types A and C

3.7.6 Planet Execution

! Message Loop: The planet message listening loop uses `crossbeam_channel` for better performance and to avoid deadlocks. Planets in stopped state will respond with `Stopped` to any message except `StartPlanetAI` or `KillPlanet`.

```rust
impl Planet {
    /// Main blocking execution loop.
    /// Called by orchestrator in separate thread.
    pub fn run(&mut self) -> Result<(), String> {
        // 1. Wait for StartPlanetAI message
        // 2. Call ai.start()
        // 3. Poll orchestrator and explorer channels using crossbeam_channel
        // 4. Dispatch to appropriate AI handlers
        // 5. Return on channel disconnection or KillPlanet message
                // 6. Send KillPlanetAck if terminated or destroyed by asteroid
    }
}
```

Listing 15: Planet Execution Loop

! Type Erasure Requirement: The `ai` field must be `Box<dyn PlanetAI>` to enable heterogeneous collections of planets. This allows the orchestrator to store different planet implementations in the same data structure.

3.7.7 Source Reference

Complete implementation: planet.rs

3.8 Explorer

Purpose: Mobile agent that travels between planets, collects resources, and interacts with planetary systems.

3.8.1 Core Responsibilities

- **Resource collection**: Gathers basic and complex resources
- **Planet navigation**: Moves between connected planets
- **Resource management**: Maintains inventory of collected resources
- **Protocol compliance**: Follows defined message protocols

3.8.2 Resource Storage

Design Note: Explorers manage resources internally without a formal Bag trait. Resource storage implementation is left to individual explorer implementations, but must support the required resource operations.

- **Reference:** `main.pdf 2.7.2`: "The bag is the set of resources... The bag could be, for example, a `Vec<BasicResource>` and a `Vec<ComplexResource>`"
- **Implementation:** Each developer team designs their own storage solution within the provided protocol constraints
- **Constraint:** The bag cannot contain energy cells, which are local to a Planet and cannot be moved

3.8.3 Key Constraints

- **No energy creation**: Cannot create or modify EnergyCells
- **Channel-based communication**: Uses provided channels for all interactions
- **Orchestrator-controlled movement**: Movement initiated through orchestrator
- **Resource transfer**: Resources move with explorer during planet transitions
- **No explorer interaction**: An Explorer cannot initiate interactions with other Explorers

! **Isolation Principle:** Explorers operate independently and cannot directly communicate with each other. All coordination must happen through the orchestrator or via shared planetary systems.

4 Message Enum Definitions

4.0.1 Message Flow Rules

- Planets in stopped state respond with `Stopped` to any message except `StartPlanetAI` or `KillPlanet`
- `KillPlanet` terminates planet execution regardless of state
- After sending `KillPlanetAck`, planet execution terminates
- `AsteroidAck` with `None` rocket indicates planet cannot defend
- **Planet destruction cascade**: When a planet is destroyed (sends `KillPlanetAck` in response to asteroid), all Explorers currently on that planet must also be terminated

4.1 Orchestrator ↔ Planet Protocol

4.1.1 Message Definitions

```rust
pub enum OrchestratorToPlanet {
    Sunray(Sunray),
    Asteroid(Asteroid),
    StartPlanetAI,
    StopPlanetAI,
    KillPlanet,  // Kill planet even when stopped
    InternalStateRequest,
    IncomingExplorerRequest {
        explorer_id: u32,
        new_mpsc_sender: mpsc::Sender<PlanetToExplorer>,
    },
    OutgoingExplorerRequest {
        explorer_id: u32,
    },
}
```

Listing 16: Orchestrator to Planet Messages

```rust
pub enum PlanetToOrchestrator {
    SunrayAck {
        planet_id: u32,
    },
    AsteroidAck {
        planet_id: u32,
        rocket: Option<Rocket>,
    },
    StartPlanetAIResult {
        planet_id: u32,
    },
    StopPlanetAIResult {
        planet_id: u32,
    },
    Stopped { planet_id: u32 },  // Response when stopped planet
       receives invalid message

    KillPlanetAck { planet_id: u32 },

    InternalStateResponse {
        planet_id: u32,
        planet_state: DummyPlanetState,
    },
    IncomingExplorerResponse {
        planet_id: u32,
        res: Result<(), String>,
    },
    OutgoingExplorerResponse {
        planet_id: u32,
        res: Result<(), String>,
    },
}
```

Listing 17: Planet to Orchestrator Messages

4.1.2 Message Response Expectations

Message	Expected Response
Sunray	SunrayAck (required)
Asteroid	AsteroidAck (with Option<Rocket>) or KillPlanetAck
StartPlanetAI	StartPlanetAIResult (acknowledgment)
StopPlanetAI	StopPlanetAIResult (acknowledgment)
KillPlanet	KillPlanetAck (acknowledgment)
Other messages (when stopped)	Stopped (default response)

Table 5: Orchestrator-Planet Response Requirements

Implementations must ensure proper response handling for all required messages.

4.2 Explorer ↔ Planet Protocol

4.2.1 Message Definitions

```
pub enum ExplorerToPlanet {
    SupportedResourceRequest { explorer_id: u32 },
    SupportedCombinationRequest { explorer_id: u32 },
    GenerateResourceRequest {
        explorer_id: u32,
        resource: BasicResourceType
    },
    CombineResourceRequest {
        explorer_id: u32,
        msg: ComplexResourceRequest
    },
    AvailableEnergyCellRequest { explorer_id: u32 },
}
```

Listing 18: Explorer to Planet Messages

```
pub enum PlanetToExplorer {
    SupportedResourceResponse {
        resource_list: HashSet<BasicResourceType>,
    },
    SupportedCombinationResponse {
        combination_list: HashSet<ComplexResourceType>,
    },
    GenerateResourceResponse {
        resource: Option<BasicResource>,
    },
    CombineResourceResponse {
        complex_response: Result<ComplexResource, (String,
            GenericResource, GenericResource)>,
    },
    AvailableEnergyCellResponse {
        available_cells: u32,
    },
}
```

Listing 19: Planet to Explorer Messages

4.3 Orchestrator ↔ Explorer Protocol

4.3.1 Message Definitions

```rust
pub enum OrchestratorToExplorer {
    StartExplorerAI,
    ResetExplorerAI,
    KillExplorerAI,
    MoveToPlanet {
        sender_to_new_planet: Option<mpsc::Sender<ExplorerToPlanet>>,
    },
    CurrentPlanetRequest,
    SupportedResourceRequest,
    SupportedCombinationRequest,
    GenerateResourceRequest { to_generate: BasicResourceType },
    CombineResourceRequest(ComplexResourceRequest),
    BagContentRequest,
    NeighborsResponse { neighbors: Vec<u32> },
}
```

Listing 20: Orchestrator to Explorer Messages

```rust
pub enum ExplorerToOrchestrator<T> {
    StartExplorerAIResult {
        explorer_id: u32,
    },
    KillExplorerAIResult {
        explorer_id: u32,
    },
    ResetExplorerAIResult {
        explorer_id: u32,
    },
    MovedToPlanetResult {
        explorer_id: u32,
    },
    CurrentPlanetResult {
        explorer_id: u32,
        planet_id: u32,
    },
    SupportedResourceResult {
        explorer_id: u32,
        supported_resources: HashSet<BasicResourceType>,
    },
    SupportedCombinationResult {
        explorer_id: u32,
        combination_list: HashSet<ComplexResourceType>,
    },
    GenerateResourceResponse {
        explorer_id: u32,
        generated: Result<(), ()>,
    },
    CombineResourceResponse {
        explorer_id: u32,
        generated: Result<(), ()>,
    },
    BagContentResponse {
```

```
        explorer_id: u32,
        bag_content: T,  // Generic type
    },
    NeighborsRequest {
        explorer_id: u32,
        current_planet_id: u32,
    },
    TravelToPlanetRequest {
        explorer_id: u32,
        current_planet_id: u32,
        dst_planet_id: u32,
    },
}
```

Listing 21: Explorer to Orchestrator Messages

4.3.2 Movement Coordination Protocol

1. Explorer sends `TravelToPlanetRequest` to orchestrator
2. Orchestrator validates movement (adjacency, permissions)
3. Orchestrator sends `MoveToPlanet` with new channel
4. Explorer connects to new planet using provided channel
5. Explorer sends `MovedToPlanetResult` to confirm
6. Orchestrator updates internal routing tables

4.4 Source Reference

Complete implementation: messages.rs

5 Protocol Interaction Stories

This section contains all sequence diagrams illustrating system interactions. Refer to Section 4 for message enum definitions.

5.1 Lifecycle Management

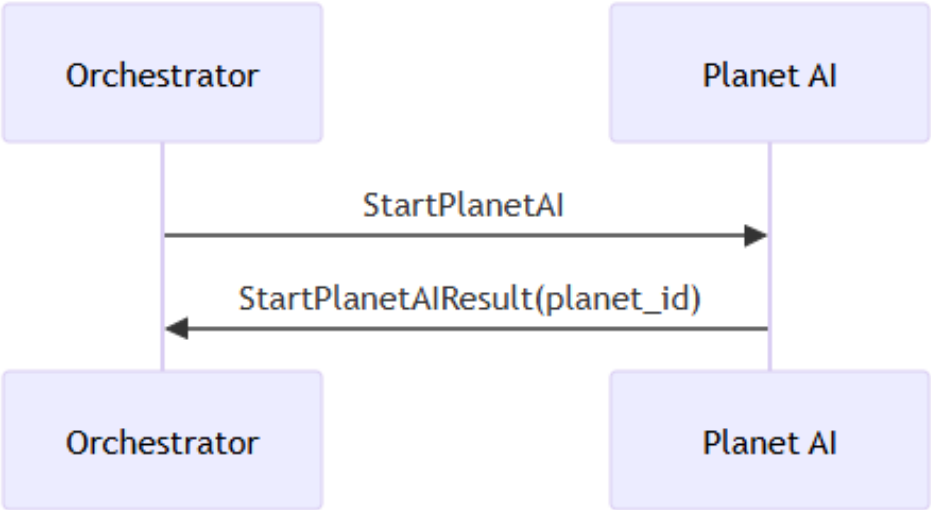

Figure 1: Planet AI initialization sequence

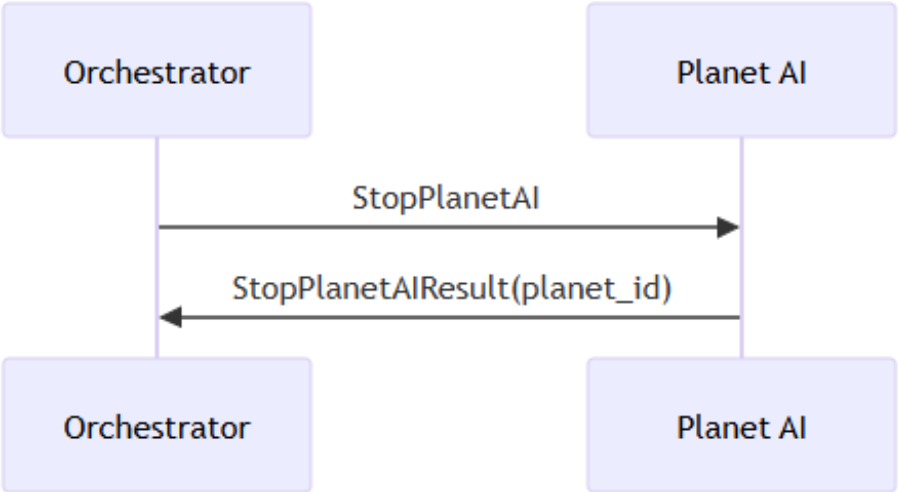

Figure 2: Planet AI termination sequence

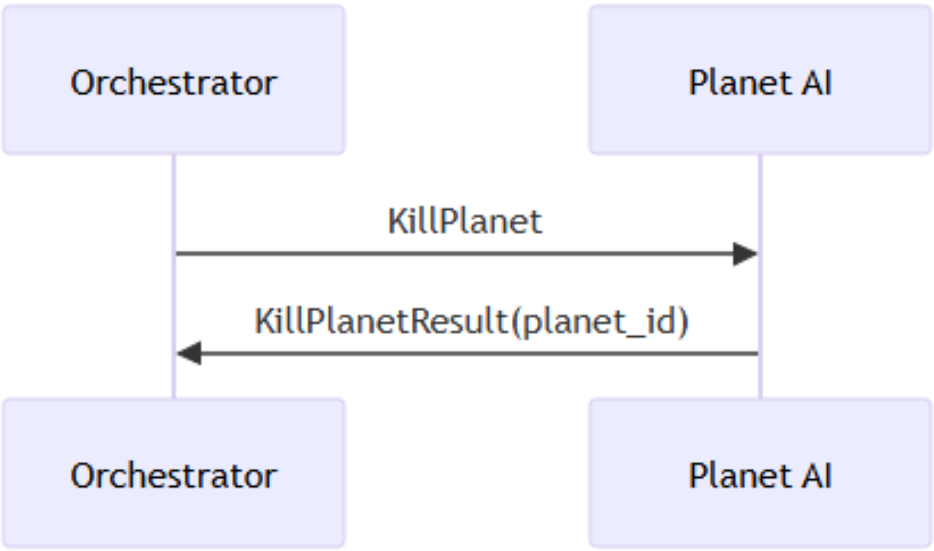

Figure 3: Planet termination via KillPlanet message (handled even when stopped)

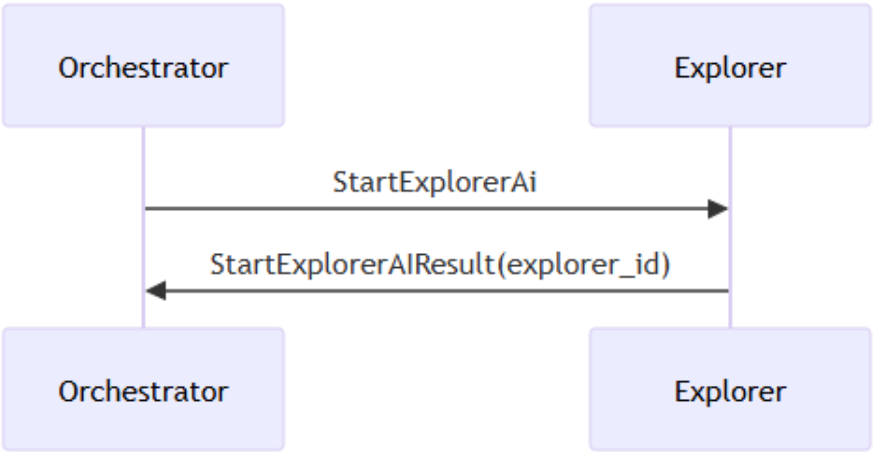

Figure 4: Explorer AI initialization sequence

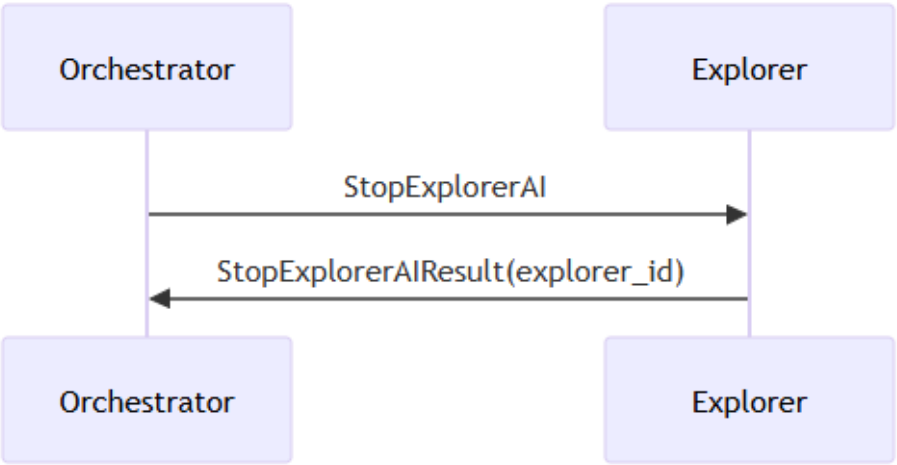

Figure 5: Explorer AI termination sequence

5.2 Sunray and Asteroid Interactions

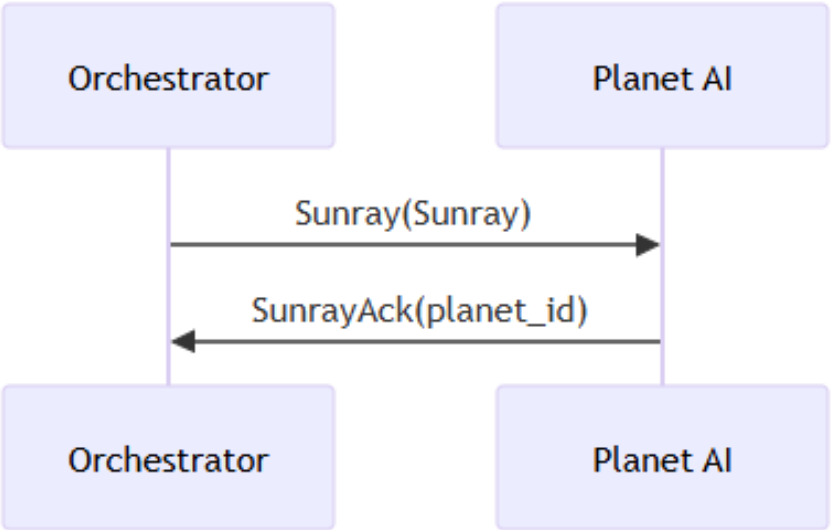

Figure 6: Sunray interaction handling by Planet AI

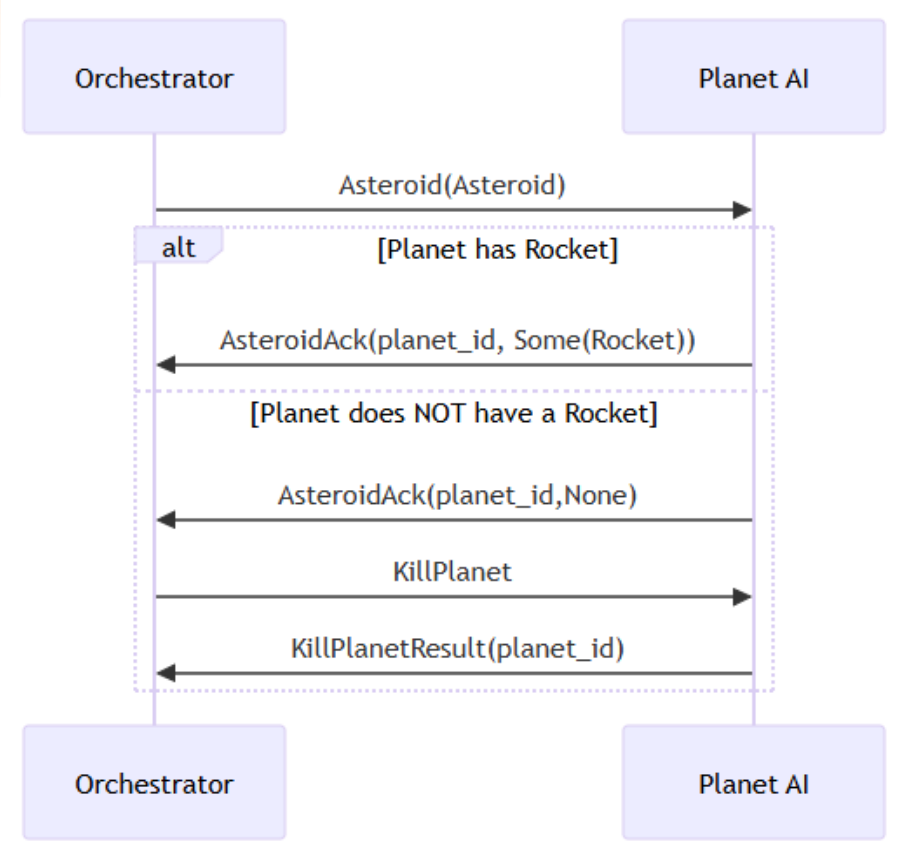

Figure 7: Asteroid defense (note: when planet is destroyed, all Explorers on it are also terminated)

5.3 Resource Operations

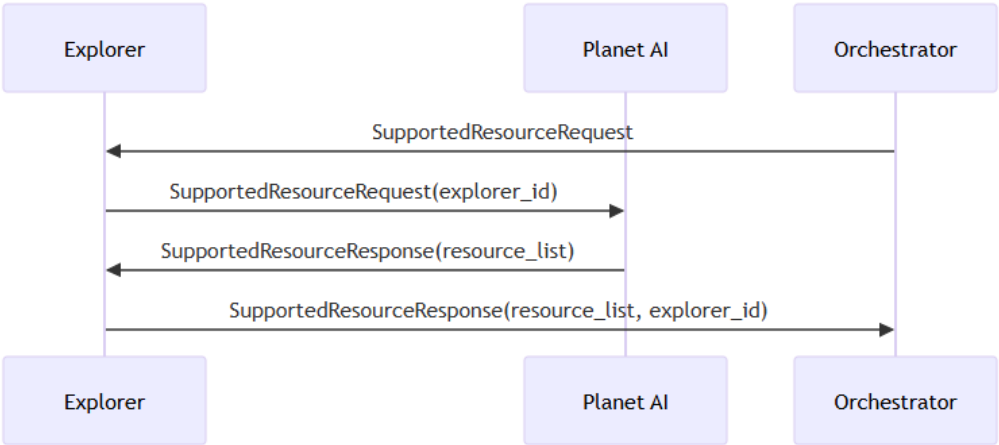

Figure 8: Basic resource discovery initiated manually by Orchestrator

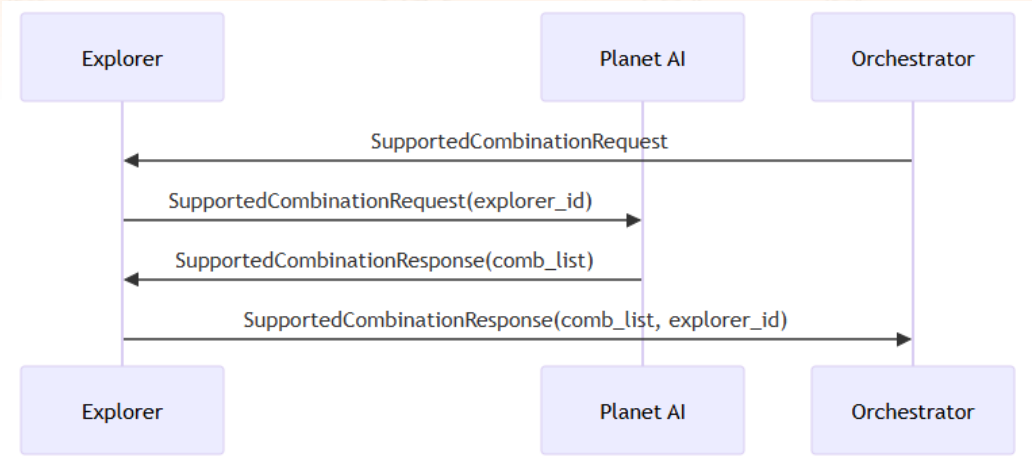

Figure 9: Combination rules discovery initiated manually by Orchestrator

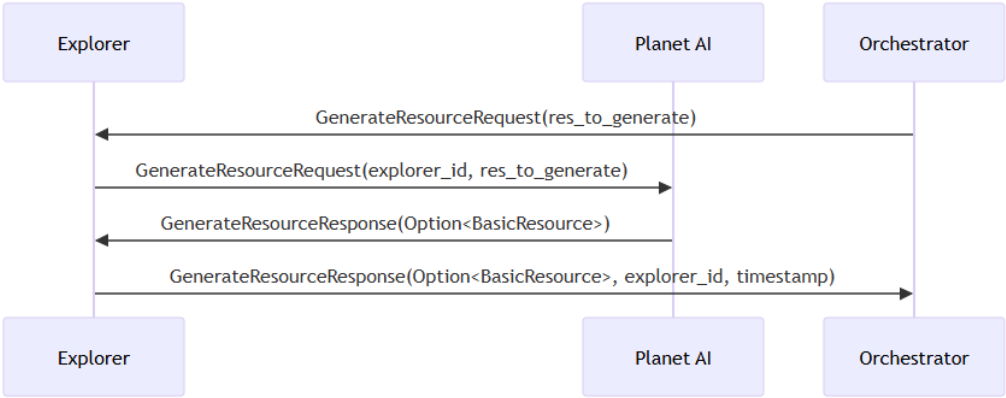

Figure 10: Basic resource generation initiated manually by Orchestrator

Figure 11: Resource combination process initiated manually by Orchestrator

Figure 12: Basic resource discovery initiated autonomously by Explorer

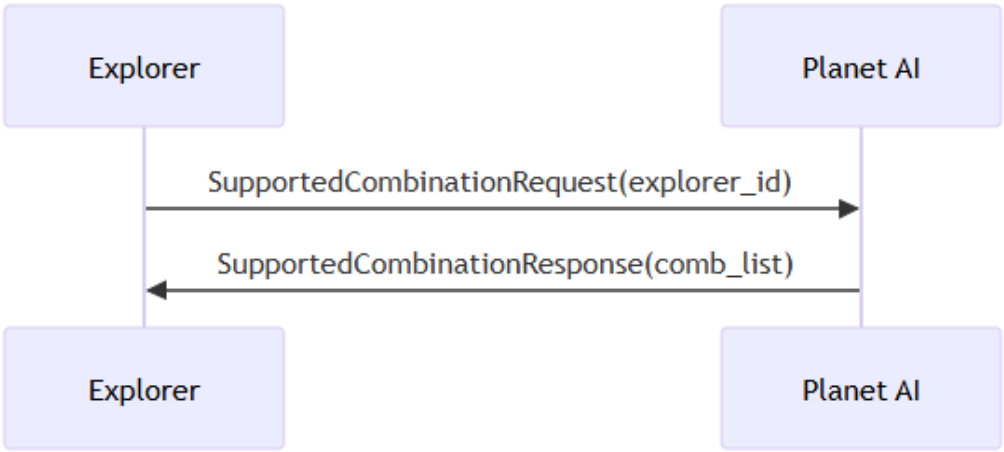

Figure 13: Combination rules discovery initiated autonomously by Explorer

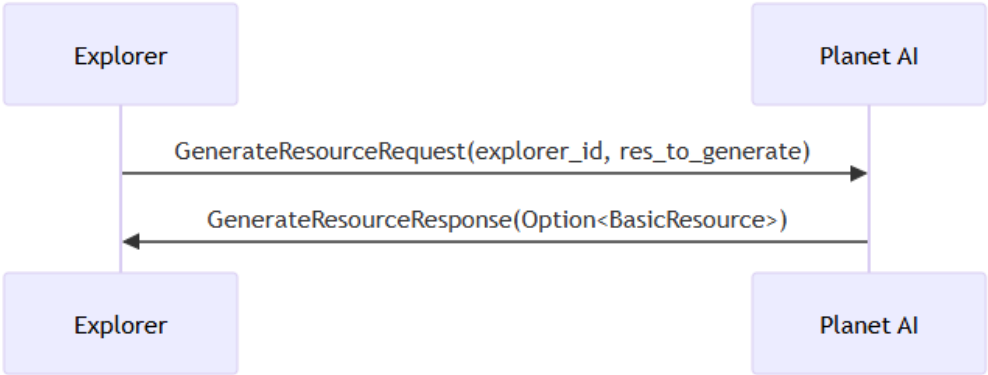

Figure 14: Basic resource generation initiated autonomously by Explorer

Figure 15: Resource combination process initiated autonomously by Explorer

5.4 Explorer Navigation

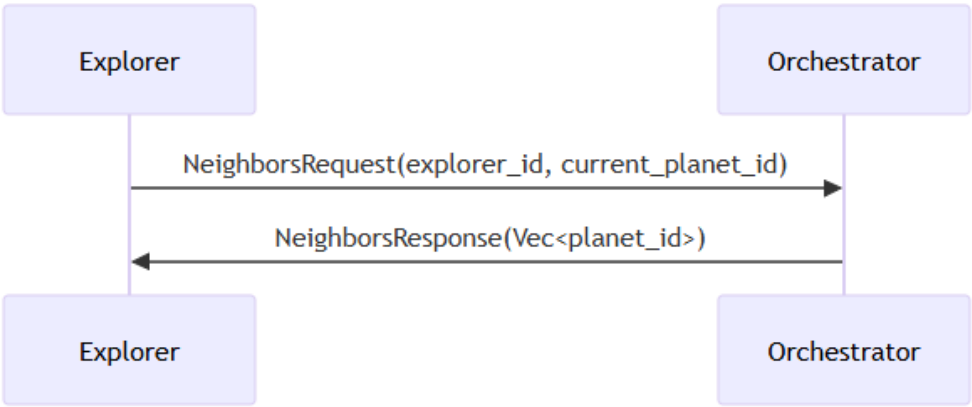

Figure 16: Neighbors discovery process for navigation planning

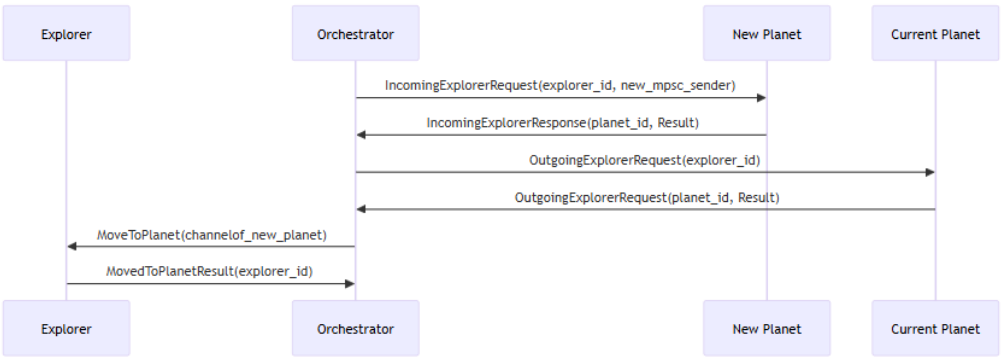

Figure 17: Manual planet movement initiated by Orchestrator

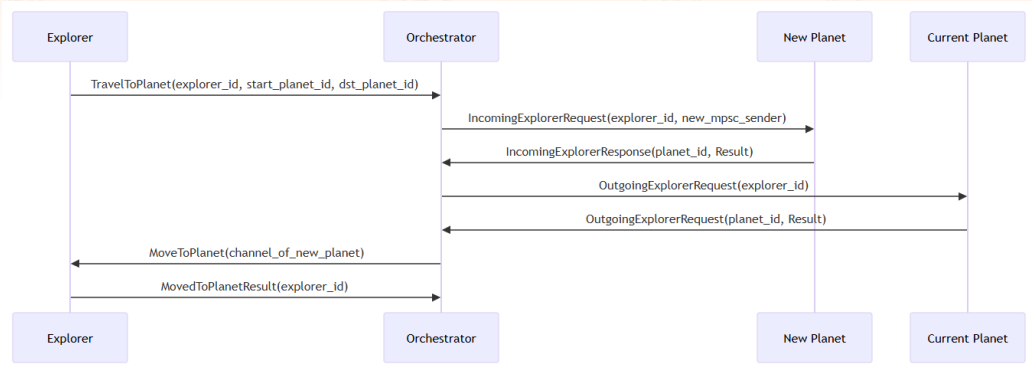

Figure 18: Autonomous planet movement initiated by Explorer

5.5 State Discovery and Inspection

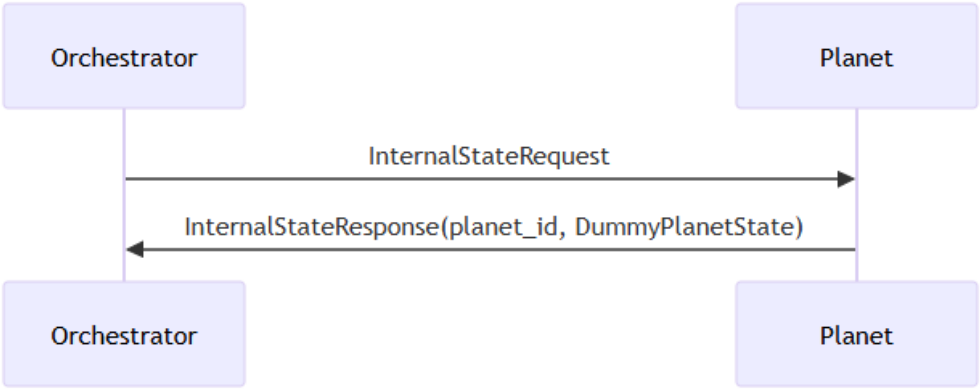

Figure 19: Internal state discovery initiated by Orchestrator

5.6 Energy Management

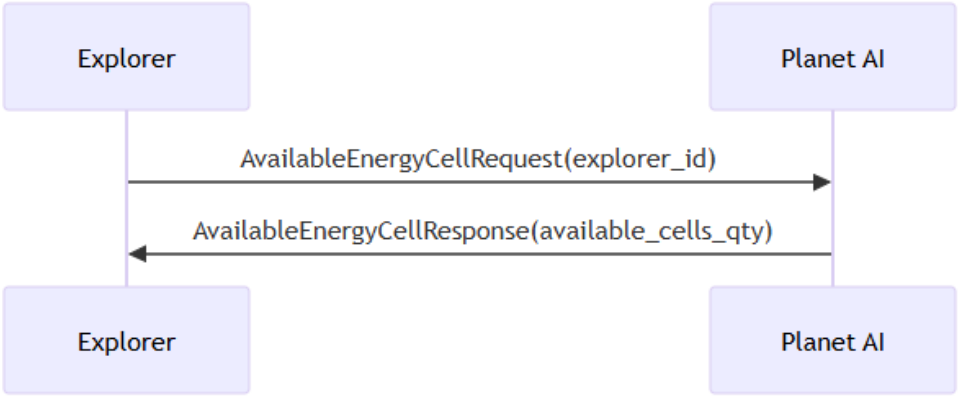

Figure 20: Energy cell availability check initiated by Explorer

5.7 Bag Contents Request

Figure 21: Bag content inspection initiated by Orchestrator

6 Observability and Diagnostics

6.1 Unified Logging Framework

All system components use a common structured logging framework for monitoring and debugging. This provides consistent, machine-readable logs across planets, explorers, and orchestrators.

The logging system enables cross-component debugging by standardizing log format and metadata. Implementers can trace message flows and diagnose issues without coordinating log formats with other teams.

6.2 Log Structure and Types

Logs capture comprehensive metadata about system events:

```rust
/// Identifies component types in the system
pub enum ActorType {
    Planet, Explorer, Orchestrator, User, Broadcast, SelfActor
}

/// Standard log severity levels
pub enum Channel { Error, Warning, Info, Debug, Trace }

/// Categories of system events
pub enum EventType {
    // Inter-component messages
    MessagePlanetToOrchestrator, MessagePlanetToExplorer,
    MessageOrchestratorToPlanet, MessageOrchestratorToExplorer,
    MessageExplorerToPlanet, MessageExplorerToOrchestrator,

    // Internal component operations
    InternalPlanetAction, InternalExplorerAction,
    InternalOrchestratorAction,

    // External interactions
    UserToPlanet, UserToExplorer, UserToOrchestrator
}

/// Structured log entry with full context
pub struct LogEvent {
    pub timestamp_unix: i64,
    pub sender_type: ActorType,
    pub sender_id: u64,
    pub receiver_type: ActorType,
    pub receiver_id: String,
    pub event_type: EventType,
    pub channel: Channel,
    pub payload: BTreeMap<String, String>,
}
```

Listing 22: Core Logging Types

Each log includes sender, receiver, timestamp, and event type. This allows reconstructing complete interaction sequences across the distributed system.

6.3 Integration with Rust's Log Ecosystem

The system integrates with the standard Rust `log` crate, allowing implementers to use familiar logging backends like `env_logger`, `simple_logger`, or `fern`.

Components emit logs through the `log` crate interface. To see output, the final binary must initialize a logger backend. Without initialization, logs are silently dropped.

6.4 Logging Requirements for Implementers

- **Use Standard Types**: Employ `ActorType`, `Channel`, `EventType` for consistency
- **Include Full Context**: Always specify sender, receiver, and event category
- **Choose Appropriate Severity**: Use `Error` for failures, `Info` for normal operations
- **Store Variable Data in Payload**: Use the key-value map for dynamic content

! Logging requires external initialization. Components cannot assume logs will be captured without a logger backend like `env_logger` being configured in the final binary.

6.5 Source Reference

Complete implementation: logging.rs

7 User-Inconstructible Types

7.1 Enforcement Mechanisms

The following types cannot be constructed by dependent crates:

Type	Construction Method	Allowed Constructor
Sunray	`pub(crate) new()`	Generator only
Asteroid	`pub(crate) new()`	Generator only
Rocket	`pub fn new(EnergyCell)`	Planets only
Complex Resources	Private constructors	Combination only
Basic Resources	Private constructors	Planet generation only
Generator	Singleton pattern	First caller only

Table 6: User-Inconstructible Types and Construction Rules

7.2 Technical Enforcement

- **Private fields**: Structs contain `_private: ()` fields
- **Pub(crate) constructors**: Construction limited to crate
- **Singleton pattern**: Generator uses static flag
- **Ownership transfer**: Resources move through defined channels
- **Trait bounds**: Operations require specific contexts

! Design Rationale: User-inconstructible types ensure system invariants by controlling object creation. This prevents invalid states and enforces protocol compliance.

8 Implementation Guidelines

8.1 For Planet Implementers

- **Implement PlanetAI trait**: Define behavior through message handlers
- **Respect planet type constraints**: Follow energy cell and recipe limits
- **Handle all required messages**: Sunray, Asteroid, Start/Stop
- **Manage energy correctly**: Charge/discharge cells as specified
- **Provide timely responses**: Respect protocol timeouts
- **Maintain thread safety**: Planets run in separate threads

8.2 For Explorer Implementers

- **Follow movement protocol**: Coordinate through orchestrator
- **Manage resource inventory**: Track collected resources
- **Handle disconnections**: Planets may become unavailable
- **Respect planet capabilities**: Check supported operations
- **Implement error recovery**: Handle failed operations gracefully

8.3 For Orchestrator Implementers

- **Own a Forge**: Not borrow, to enforce singleton
- **Manage channel lifetimes**: Coordinate component connections
- **Enforce protocols**: Validate message sequences
- **Handle failures**: Manage component crashes
- **Maintain system state**: Track planet and explorer status

8.4 Common Requirements

- **No panics**: Use Result types for recoverable errors
- **Comprehensive logging**: Use structured logging with IDs
- **Thread safety**: Components communicate via channels
- **Protocol compliance**: Follow defined message sequences
- **Resource management**: Properly transfer ownership

9 Source Code References

Complete implementations are available in the common crate:

Component	Source File
Forge	forge.rs
Sunray	sunray.rs
Asteroid	asteroid.rs
EnergyCell	energy_cell.rs
Rocket	rocket.rs
Resources	resource.rs
Planet	planet.rs
Protocols	messages.rs
Logging	logging.rs

Table 7: Source File Locations

Repository Access: The complete source code is available at the project repository. This documentation focuses on public interfaces and protocols; implementation details are in the source files.

10 Protocol Summary

10.1 Key Invariants

- **Singleton Generator**: Only one Generator exists system-wide
- **Energy conservation**: Operations consume charged EnergyCells
- **Ownership transfer**: Resources move through channels
- **Timely responses**: Components respect protocol timeouts
- **State consistency**: Components maintain valid states

10.2 Error Handling Philosophy

- **Recoverable errors**: Return resources when possible
- **Clean failures**: No panics, proper error reporting
- **State preservation**: Failed operations don't corrupt state
- **Resource return**: Failed combinations return inputs

10.3 Extension Points

The system is designed for extension:

- New planet types can implement PlanetAI trait
- New resources can be added to resource system
- New protocols can extend message enums
- Custom explorers can implement movement logic
- Additional components can integrate via channels

For External Implementers: This documentation provides everything needed to create compatible planets and explorers. Focus on implementing the defined traits and following the protocols. The source code provides reference implementations.